

École Marocaine des Sciences de l'Ingénieur

Filière : **Ingénierie Informatique et Réseaux**

Option : **Intelligence Artificielle et Data Science**

4^{ème} année Génie Informatique

Module : **Deep Learning**

Rapport de Projet de Fin de Module

FootAI

Modèles de Deep Learning pour données tabulaires, images et séquences

Réalisé par :

Taha ZAAMI

Encadré par :

Pr. BATAL Mossab

Année Universitaire 2025–2026

Résumé

Ce rapport présente une étude comparative de trois familles fondamentales du Deep Learning : le perceptron multicouche (MLP), les réseaux convolutifs (CNN) et les modèles séquentiels RNN/LSTM/GRU avec une extension Seq2Seq. L'objectif est de montrer que le choix d'une architecture n'est pas arbitraire : il dépend de la structure des données étudiées. Le MLP est appliqué à des données tabulaires issues des tirs de football StatsBomb, le CNN est appliqué à des crops d'images extraits d'un dataset Roboflow/YOLO, et les modèles récurrents sont appliqués à un corpus textuel footballistique construit à partir d'événements de match.

La démarche adoptée suit une logique théorique et expérimentale. Chaque chapitre présente d'abord les fondements mathématiques, puis le cadre expérimental, les résultats obtenus avec PyTorch, leur interprétation critique et une synthèse finale. Les résultats montrent que le MLP reste pertinent pour des vecteurs tabulaires bien préparés, que les CNN exploitent efficacement la structure spatiale des images, et que les modèles RNN/LSTM/GRU permettent de traiter des séquences grâce à une mémoire temporelle. Enfin, la partie Seq2Seq illustre la génération de phrases footballistiques simples avec décodage glouton et beam search.

Table des matières

Résumé Exécutif	1
Introduction Générale	6
I Le Perceptron Multicouche (MLP)	7
I.1 Fondements Théoriques et Mathématiques	7
I.1.1 Architecture et propagation avant	7
I.1.2 Fonction de perte et optimisation	7
I.1.3 Rétropropagation et paramètres PyTorch	7
I.1.4 Stratégies d'initialisation	8
I.2 Cadre Expérimental	8
I.3 Étude des Stratégies d'Initialisation	8
I.4 Comparaison : nn.Sequential vs Classe Personnalisée	9
I.5 Évaluation Finale sur le Test Set	9
I.6 Synthèse du Chapitre I	10
II Réseaux de Neurones Convolutifs (CNN)	11
II.1 Fondements Théoriques et Mathématiques	11
II.1.1 Limites du MLP sur les images	11
II.1.2 Corrélation croisée 2D	11
II.1.3 Formule dimensionnelle	11
II.1.4 Pooling et convolution 1×1	11
II.2 Cadre Expérimental	12
II.3 Étude des Variantes Architecturales	12
II.4 Visualisation des Feature Maps	13
II.5 Évaluation Finale	13
II.6 Synthèse du Chapitre II	14
III Réseaux Récurrents, LSTM, GRU et Seq2Seq	15
III.1 Fondements Théoriques et Mathématiques	15
III.1.1 Limites des architectures statiques face aux séquences	15
III.1.2 RNN simple	15
III.1.3 BPTT et gradient clipping	15

III.1.4 LSTM et GRU	15
III.1.5 Architecture Seq2Seq	16
III.2 Cadre Expérimental	16
III.3 Comparaison des Cellules Récurentes	16
III.4 Entraînement et Inférence du Modèle Seq2Seq	17
III.5 Évaluation BLEU et Stratégies de Décodage	18
III.6 Synthèse du Chapitre III	18
IV Comparaison Croisée et Conclusion Générale	20
IV.1 Tableau Comparatif Global des Trois Architectures	20
IV.2 Le Biais Inductif : Clé de la Réussite en Deep Learning	20
IV.3 Perspectives d'Amélioration	20
IV.4 Conclusion Générale	21
Références Bibliographiques	22

Table des figures

1	Courbes d'entraînement du meilleur MLP	9
2	Matrice de confusion du meilleur MLP sur le test set	10
1	Distribution des crops par classe dans le dataset image	12
2	Visualisation de cartes de caractéristiques apprises par la première convolution du CNN	13
3	Matrice de confusion du meilleur modèle CNN	14
1	Matrice de confusion du meilleur modèle de classification séquentielle	17
2	Courbes d'entraînement du modèle Seq2Seq GRU	18

Liste des tableaux

1	Paramètres expérimentaux du Chapitre I – MLP / StatsBomb	8
2	Résultats comparatifs des initialisations du MLP personnalisé	8
3	Comparaison entre le MLP <code>nn.Sequential</code> et le meilleur MLP personnalisé . .	9
1	Paramètres expérimentaux du Chapitre II – CNN / Roboflow YOLO	12
2	Résultats comparatifs des modèles image sur le test set	13
1	Paramètres expérimentaux du Chapitre III – RNN/LSTM/GRU/Seq2Seq	16
2	Comparaison RNN, LSTM et GRU sur la classification de séquences	16
3	Scores BLEU moyens du modèle Seq2Seq	18
1	Comparaison globale des architectures étudiées dans FootAI	20

Introduction Générale

L'apprentissage profond ne repose pas sur un modèle unique capable de traiter efficacement toutes les données. Au contraire, il s'appuie sur des architectures spécialisées qui exploitent des hypothèses différentes sur la structure des observations. Ces hypothèses sont appelées biais inductifs. Un MLP traite une observation comme un vecteur de caractéristiques, un CNN suppose une organisation spatiale locale, tandis qu'un RNN exploite l'ordre temporel d'une séquence.

Dans ce projet de fin de module, le fil conducteur choisi est le football. Cette thématique permet de manipuler trois types de données complémentaires : des données tabulaires décrivant des tirs, des images issues de vidéos ou d'annotations YOLO, et des séquences textuelles décrivant des événements. Le projet permet ainsi de relier les notions théoriques du module de Deep Learning à une implémentation pratique sous PyTorch.

La problématique générale peut être formulée comme suit : comment adapter le choix d'une architecture de Deep Learning à la nature des données étudiées, et comment interpréter expérimentalement les performances obtenues ? Pour répondre à cette question, le rapport est structuré en quatre chapitres. Le premier chapitre traite le MLP sur données tabulaires. Le deuxième chapitre étudie les CNN pour les images. Le troisième chapitre analyse les modèles RNN, LSTM, GRU et Seq2Seq pour les séquences. Le quatrième chapitre propose une comparaison croisée et une conclusion générale.

Chapitre I – Le Perceptron Multicouche (MLP)

I.1 – Fondements Théoriques et Mathématiques

I.1.1 – Architecture et propagation avant

Un perceptron multicouche est un réseau feed-forward composé d'une couche d'entrée, d'une ou plusieurs couches cachées et d'une couche de sortie. Pour une couche l , la propagation avant s'écrit :

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \sigma(z^{(l)}).$$

Dans ce projet, les couches cachées utilisent principalement l'activation ReLU :

$$\text{ReLU}(z) = \max(0, z).$$

La sortie correspond à une classification multiclasse des tirs en trois niveaux : *faible*, *moyen* et *élevé*. Contrairement à un CNN ou à un RNN, le MLP n'intègre aucun biais spatial ou temporel : il apprend uniquement à partir des variables tabulaires fournies.

I.1.2 – Fonction de perte et optimisation

Pour une classification multiclasse, la fonction de perte utilisée est la Cross-Entropy Loss. Elle compare la distribution prédite par le modèle à la classe réelle :

$$\mathcal{L}(y, \hat{y}) = - \sum_{c=1}^C y_c \log(\hat{y}_c).$$

L'optimisation est réalisée avec Adam, qui combine une moyenne mobile du gradient et une moyenne mobile du carré du gradient afin d'adapter le taux d'apprentissage à chaque paramètre. Ce choix est adapté aux petits datasets tabulaires car il accélère souvent la convergence et stabilise l'entraînement.

I.1.3 – Rétropropagation et paramètres PyTorch

Dans PyTorch, un modèle est défini comme une classe héritant de `nn.Module` ou comme une séquence de couches avec `nn.Sequential`. Les poids et biais sont enregistrés comme paramètres apprenables. Après un passage avant, l'appel à `loss.backward()` calcule les gradients, puis l'optimiseur met à jour les paramètres.

La bonne pratique consiste à sauvegarder uniquement le `state_dict` du modèle, c'est-à-dire le dictionnaire des tenseurs de poids et de biais. Cela permet de recharger le modèle dans une

architecture identique sans sérialiser directement le code Python.

I.1.4 – Stratégies d'initialisation

L'initialisation influence fortement l'apprentissage. Trois stratégies ont été testées :

- **Gaussienne** : poids tirés selon une loi normale de faible variance ;
- **Constante** : tous les poids démarrent avec une valeur identique ;
- **Xavier** : variance adaptée au nombre d'entrées et de sorties afin de stabiliser la propagation du signal.

L'initialisation constante est utile pédagogiquement mais mauvaise en pratique, car elle provoque un problème de symétrie : plusieurs neurones évoluent de manière identique.

I.2 – Cadre Expérimental

Le dataset utilisé est construit à partir de StatsBomb Open Data. Chaque observation représente un tir de football décrit par des variables numériques et catégorielles. Les variables sont nettoyées, encodées puis normalisées avant l'entraînement.

TABEAU 1 – Paramètres expérimentaux du Chapitre I – MLP / StatsBomb

Paramètre	Valeur / Description
Dataset	StatsBomb Open Data – tirs footballistiques
Échantillons	228 tirs exploitables
Variables d'entrée	29 caractéristiques après encodage et normalisation
Classes	3 niveaux : faible, moyen, élevé
Modèles	MLP avec <code>nn.Sequential</code> et MLP personnalisé <code>nn.Module</code>
Initialisations	Gaussienne, Xavier, constante
Device	GPU CUDA disponible durant l'entraînement
Métriques	Accuracy, précision, rappel, F1-score et matrice de confusion

I.3 – Étude des Stratégies d'Initialisation

TABEAU 2 – Résultats comparatifs des initialisations du MLP personnalisé

Initialisation	Loss	Accuracy	Précision	Rappel	F1
Gaussienne	1.088	60.87%	59.82%	60.87%	58.06%
Xavier	0.988	50.00%	49.55%	50.00%	49.74%
Constante	1.067	56.52%	38.11%	56.52%	45.49%

Analyse et interprétation

L'initialisation gaussienne obtient le meilleur F1-score dans cette expérience. L'initialisation constante reste limitée car elle ne diversifie pas suffisamment les neurones au départ. Le résultat de Xavier n'est pas le meilleur ici, ce qui montre qu'une stratégie théoriquement robuste peut être sensible à la taille du dataset, au découpage train/test et aux hyperparamètres retenus. L'expérience reste donc à interpréter en lien avec le contexte expérimental.

I.4 – Comparaison : `nn.Sequential` vs Classe Personnalisée

Le modèle `nn.Sequential` sert à construire rapidement une chaîne linéaire de couches. La classe personnalisée héritant de `nn.Module` offre davantage de flexibilité, notamment pour les architectures non linéaires, les branchements, les connexions résiduelles ou les modèles encodeur-décodeur.

TABEAU 3 – Comparaison entre le MLP `nn.Sequential` et le meilleur MLP personnalisé

Modèle	Loss	Accuracy	Précision	Rappel	F1
MLP <code>nn.Sequential</code>	0.897	69.57%	70.31%	69.57%	69.80%
MLP personnalisé	1.088	60.87%	59.82%	60.87%	58.06%

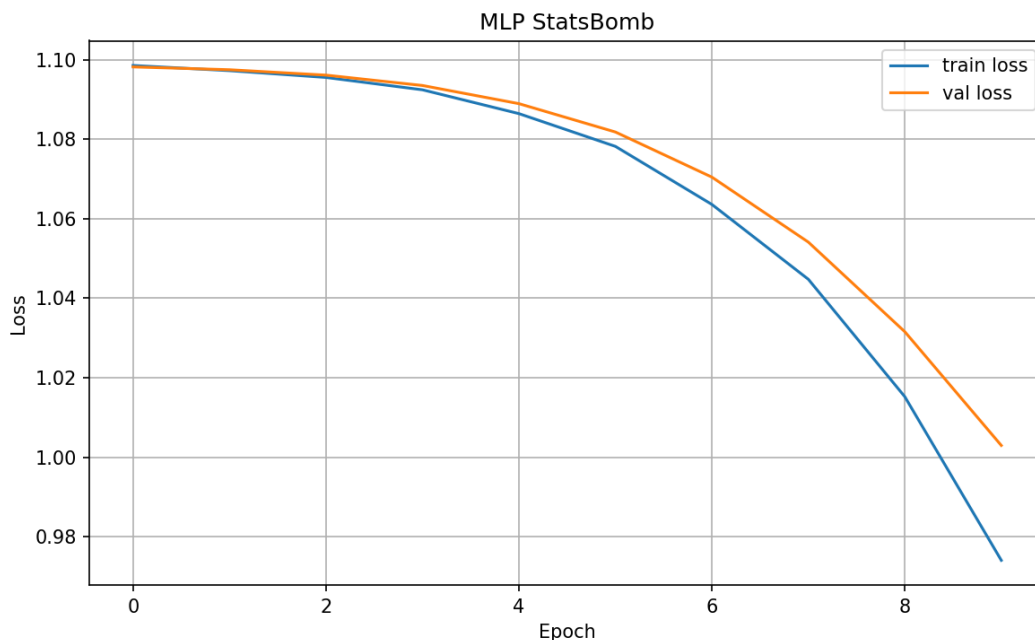


FIGURE 1 – Courbes d'entraînement du meilleur MLP

I.5 – Évaluation Finale sur le Test Set

Le meilleur modèle rechargé par `state_dict` obtient une accuracy d'environ 56.52%, une précision de 56.97%, un rappel de 56.52% et un F1-score de 56.71%. Les résultats restent modestes

par rapport aux parties image et séquence, ce qui s'explique par la faible taille du dataset de tirs et par la difficulté de prédire une classe de qualité à partir d'un nombre limité de variables.

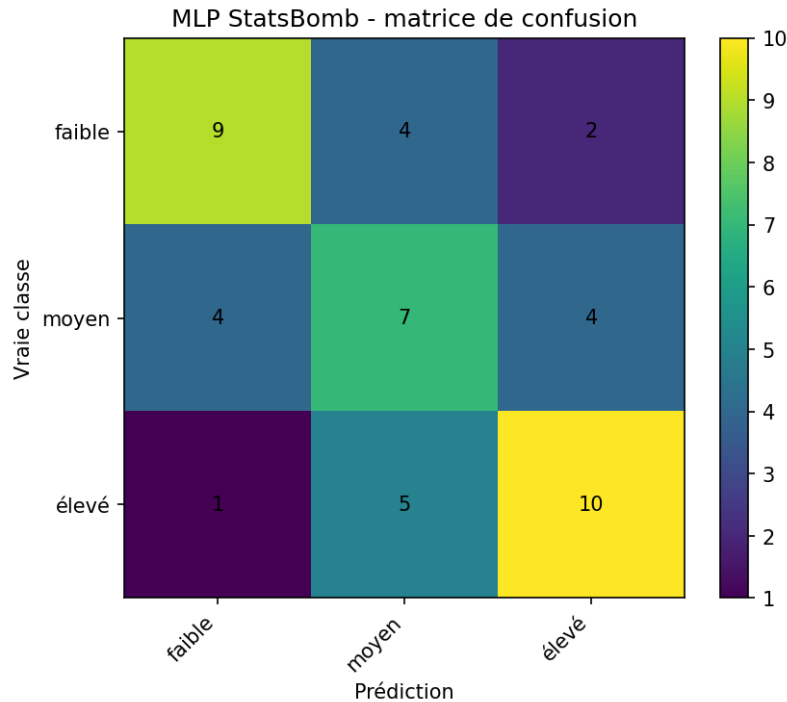


FIGURE 2 – Matrice de confusion du meilleur MLP sur le test set

Analyse et interprétation

Le MLP constitue une solution pertinente pour des données tabulaires, mais ses performances dépendent fortement de la qualité des variables construites. Ici, les tirs de football contiennent une information contextuelle partielle : la position, le type d'action ou la situation de jeu ne suffisent pas toujours à expliquer la classe finale. Pour améliorer le modèle, il serait utile d'ajouter des variables comme l'angle de tir, la pression défensive, la distance au but, le type d'assistance ou le contexte temporel du match.

I.6 – Synthèse du Chapitre I

- Le MLP est adapté aux données tabulaires préparées sous forme de vecteurs.
- `nn.Sequential` est simple et efficace pour un pipeline linéaire.
- `nn.Module` devient indispensable pour les architectures plus flexibles.
- L'initialisation et la normalisation influencent fortement la stabilité de l'apprentissage.
- Les limites observées proviennent surtout de la taille du dataset et de la richesse des variables.

Chapitre II – Réseaux de Neurones Convolutifs (CNN)

II.1 – Fondements Théoriques et Mathématiques

II.1.1 – Limites du MLP sur les images

Un MLP appliqué à une image doit d'abord aplatir celle-ci en un vecteur. Cette opération détruit la topologie spatiale : deux pixels voisins dans l'image peuvent devenir simplement deux valeurs indépendantes dans le vecteur. Le MLP doit donc apprendre les relations spatiales sans biais inductif adapté.

II.1.2 – Corrélation croisée 2D

Un CNN conserve la structure spatiale grâce à des filtres convolutifs. En pratique, PyTorch implémente une corrélation croisée :

$$S(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n).$$

Le même noyau est appliqué à toutes les positions de l'image. Ce partage des poids permet de détecter le même motif, par exemple un contour ou une texture, à plusieurs endroits.

II.1.3 – Formule dimensionnelle

La taille de sortie d'une convolution ou d'un pooling est calculée par :

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1,$$

où I est la taille d'entrée, K la taille du noyau, P le padding et S le stride. Cette formule est essentielle pour construire correctement les architectures CNN et éviter les erreurs de dimensions.

II.1.4 – Pooling et convolution 1×1

Le max-pooling réduit les dimensions spatiales tout en conservant les activations les plus fortes. Il introduit une invariance locale aux petits déplacements. La convolution 1×1 agit comme une combinaison linéaire des canaux et permet d'ajouter de la non-linéarité ou de modifier le nombre de canaux sans changer la résolution spatiale.

II.2 – Cadre Expérimental

La partie image utilise un dataset Roboflow téléchargé au format YOLOv8. Les annotations sont transformées en crops d'objets, puis ces crops sont classés en six catégories liées au football : ballon, gardien, arbitre principal, joueur, arbitre assistant et staff.

TABEAU 1 – Paramètres expérimentaux du Chapitre II – CNN / Roboflow YOLO

Paramètre	Valeur / Description
Dataset	Roboflow workspace project11-vwk21, projet x_321_5star, version 8
Format source	YOLOv8 avec bounding boxes
Crops générés	4 200 images équilibrées
Classes	Ball, Goalkeeper, Main_referee, Player, Side_referee, Staff_members
Split	2 940 train, 630 validation, 630 test
Taille image	64 × 64 pixels
Batch size	32
Architectures	MLP image aplatie, CNN baseline et variantes LeNet

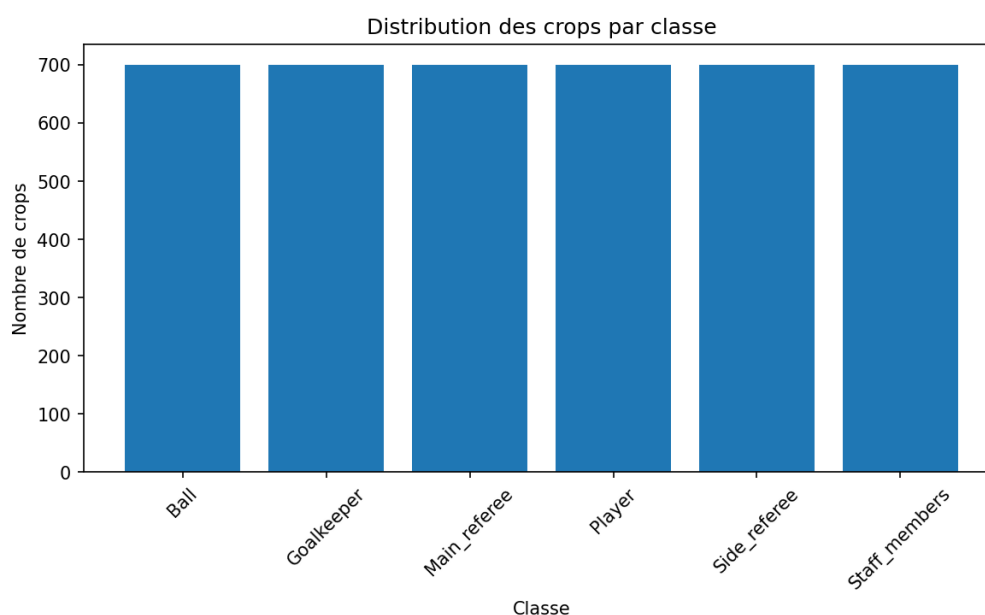


FIGURE 1 – Distribution des crops par classe dans le dataset image

II.3 – Étude des Variantes Architecturales

Plusieurs variantes de CNN ont été testées afin d'observer l'influence du padding, du stride, du pooling, du nombre de filtres et de la convolution 1×1 .

TABEAU 2 – Résultats comparatifs des modèles image sur le test set

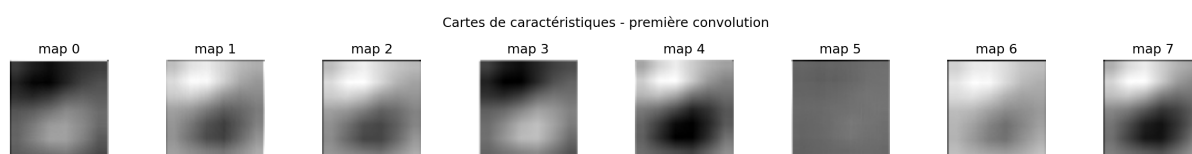
Modèle	Loss	Accuracy	Précision	Rappel	F1
CNN plus filtres	0.448	86.19%	86.50%	86.19%	86.10%
CNN baseline	0.489	85.71%	86.34%	85.71%	85.85%
CNN sans padding	0.488	84.29%	84.35%	84.29%	84.22%
CNN stride 2	0.523	84.13%	84.22%	84.13%	84.11%
CNN average pooling	0.509	84.13%	84.72%	84.13%	84.05%
CNN conv 1×1	0.559	81.90%	83.04%	81.90%	81.78%
MLP image aplatie	0.827	75.40%	76.49%	75.40%	75.59%

Analyse et interprétation

Le meilleur modèle est le CNN avec plus de filtres. Cette amélioration s'explique par une capacité plus élevée à apprendre des motifs variés : texture du ballon, silhouettes des joueurs, couleurs des tenues ou formes des arbitres. Le MLP image aplatie est nettement inférieur, ce qui confirme qu'il perd une partie de l'information spatiale. Le stride de 2 réduit rapidement la résolution et peut supprimer des détails importants, surtout pour de petits objets comme le ballon.

II.4 – Visualisation des Feature Maps

Les cartes de caractéristiques permettent d'observer ce que les premières couches du CNN apprennent. Les couches proches de l'entrée détectent généralement des primitives visuelles : contours, zones contrastées, couleurs ou textures. Les couches suivantes combinent ces primitives pour construire des représentations plus abstraites.

**FIGURE 2** – Visualisation de cartes de caractéristiques apprises par la première convolution du CNN

II.5 – Évaluation Finale

Le meilleur modèle, `CNN_plus_filtres`, obtient une accuracy de 86.19% et un F1-score pondéré d'environ 86.10%. La matrice de confusion montre que les classes les plus reconnaissables, comme le ballon, atteignent de bons scores, tandis que certaines classes visuellement proches, notamment staff et arbitres, restent plus difficiles à distinguer.

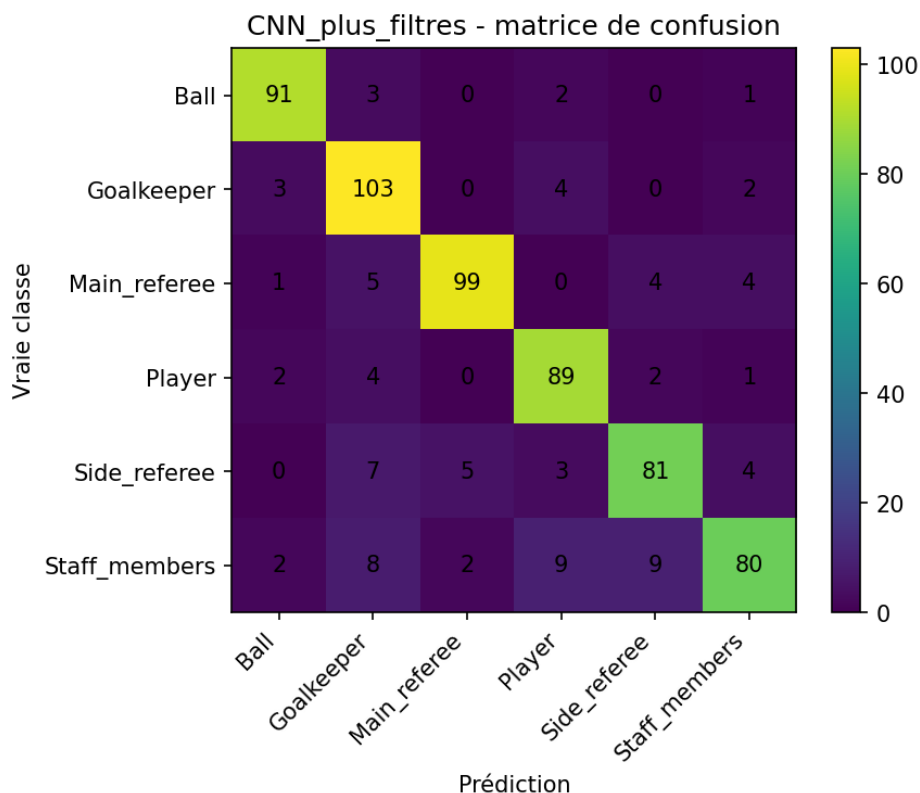


FIGURE 3 – Matrice de confusion du meilleur modèle CNN

Analyse et interprétation

La partie CNN valide clairement l'intérêt du biais spatial. Le CNN exploite la localité, le partage des poids et la hiérarchie de représentations. L'amélioration par rapport au MLP image montre que la structure de l'architecture compte autant que le nombre de paramètres. Les limites restantes viennent principalement de la qualité des crops, de la proximité visuelle entre certaines classes et de la taille relativement réduite des images.

II.6 – Synthèse du Chapitre II

- Les CNN sont plus pertinents que les MLP pour les images car ils conservent la structure spatiale.
- Le padding, le stride et le pooling modifient directement la quantité d'information conservée.
- Augmenter le nombre de filtres améliore la capacité du modèle mais augmente le coût.
- Les feature maps confirment l'apprentissage progressif de représentations visuelles.
- Le meilleur modèle atteint environ 86% de F1-score sur six classes footballistiques.

Chapitre III – Réseaux Récurrents, LSTM, GRU et Seq2Seq

III.1 – Fondements Théoriques et Mathématiques

III.1.1 – Limites des architectures statiques face aux séquences

Le MLP et le CNN ne modélisent pas directement la dépendance temporelle entre les éléments d'une séquence. Dans un texte footballistique, l'ordre des mots modifie le sens. Par exemple, une séquence décrivant une passe réussie doit être comprise différemment d'une séquence décrivant un tir ou un duel. Il faut donc un modèle capable de conserver une mémoire du passé.

III.1.2 – RNN simple

Un RNN introduit un état caché h_t qui dépend de l'entrée courante x_t et de l'état précédent h_{t-1} :

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h).$$

Le même ensemble de poids est partagé à tous les pas de temps. Ce partage temporel permet de traiter des séquences de longueur variable.

III.1.3 – BPTT et gradient clipping

L'entraînement d'un RNN se fait par rétropropagation à travers le temps (BPTT). Le gradient traverse plusieurs pas temporels, ce qui peut conduire à des gradients évanescents ou explosifs. Pour limiter l'explosion, on utilise le gradient clipping :

$$g \leftarrow \min \left(1, \frac{\theta}{\|g\|} \right) g.$$

En pratique, l'appel `torch.nn.utils.clip_grad_norm_` est une ligne importante dans l'entraînement des modèles récurrents.

III.1.4 – LSTM et GRU

Le LSTM ajoute une cellule mémoire et des portes d'oubli, d'entrée et de sortie. Le GRU simplifie cette structure avec une porte de mise à jour et une porte de réinitialisation. Dans ce projet, les trois modèles RNN, LSTM et GRU sont comparés sur une tâche de classification d'événements textuels.

III.1.5 – Architecture Seq2Seq

Le modèle Seq2Seq est composé d'un encodeur et d'un décodeur. L'encodeur lit la séquence source et produit un état de contexte. Le décodeur génère la séquence cible token par token. Pendant l'entraînement, le teacher forcing fournit au décodeur le vrai token précédent. Pendant l'inférence, deux stratégies sont testées : le décodage glouton et le beam search.

III.2 – Cadre Expérimental

Le corpus de la partie III est construit à partir de StatsBomb Open Data transformé en phrases simples. Deux tâches sont considérées : classification de séquences footballistiques et génération de phrases par Seq2Seq.

TABEAU 1 – Paramètres expérimentaux du Chapitre III – RNN/LSTM/GRU/Seq2Seq

Paramètre		Valeur / Description
Dataset		StatsBomb Open Data transformé en corpus textuel
Phrases classification		2 500 phrases
Paires Seq2Seq		1 500 paires source/cible
Vocabulaire classification		614 tokens
Vocabulaire Seq2Seq	source	32 tokens
Vocabulaire Seq2Seq	cible	52 tokens
Modèles classification		RNN, LSTM, GRU
Modèle génération		Encodeur-décodeur GRU
Métriques		Accuracy, F1-score, matrice de confusion, BLEU

III.3 – Comparaison des Cellules Récurrentes

TABEAU 2 – Comparaison RNN, LSTM et GRU sur la classification de séquences

Modèle	Loss	Accuracy	Précision	Rappel	F1
RNN	0.0029	100%	100%	100%	100%
LSTM	0.0143	100%	100%	100%	100%
GRU	0.0169	100%	100%	100%	100%

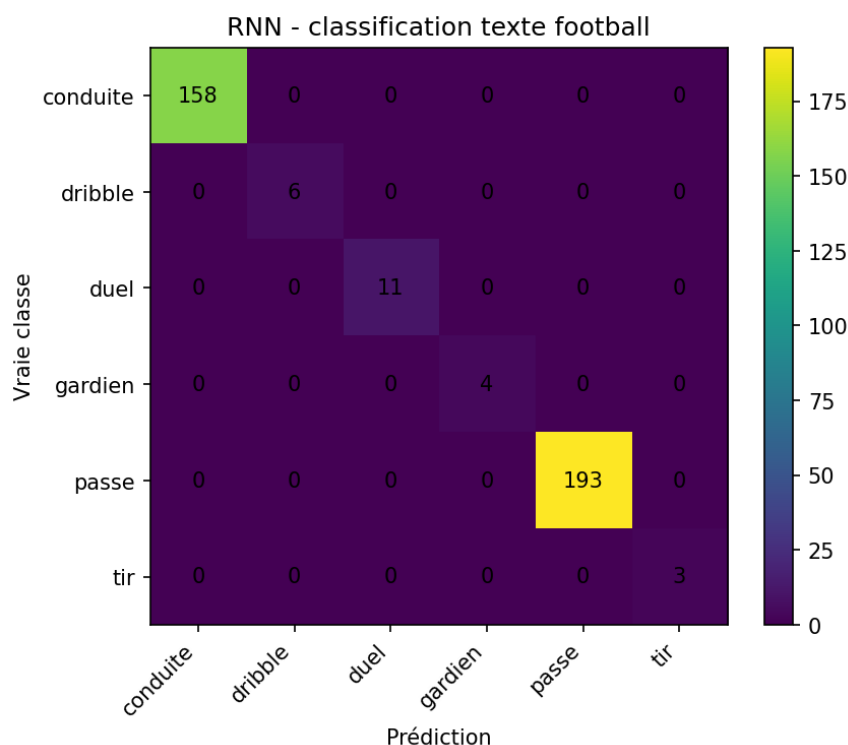


FIGURE 1 – Matrice de confusion du meilleur modèle de classification séquentielle

Analyse et interprétation

Les trois modèles atteignent 100% sur la tâche de classification. Ce résultat ne doit pas être interprété comme une supériorité absolue des modèles, mais comme un signe que le corpus généré contient des structures lexicales très régulières. Le RNN simple suffit lorsque les séquences sont courtes et fortement déterministes. Sur des textes naturels plus longs, le LSTM ou le GRU deviendraient plus robustes.

III.4 – Entraînement et Inférence du Modèle Seq2Seq

Le modèle Seq2Seq GRU apprend à transformer une description source simplifiée en phrase cible en français. Par exemple, une source telle que *pass ground pass successful* peut être transformée en une phrase explicative comme *le joueur réussit une passe*. La courbe d'entraînement montre une baisse rapide de la perte, ce qui indique que le modèle apprend la correspondance entre source et cible.

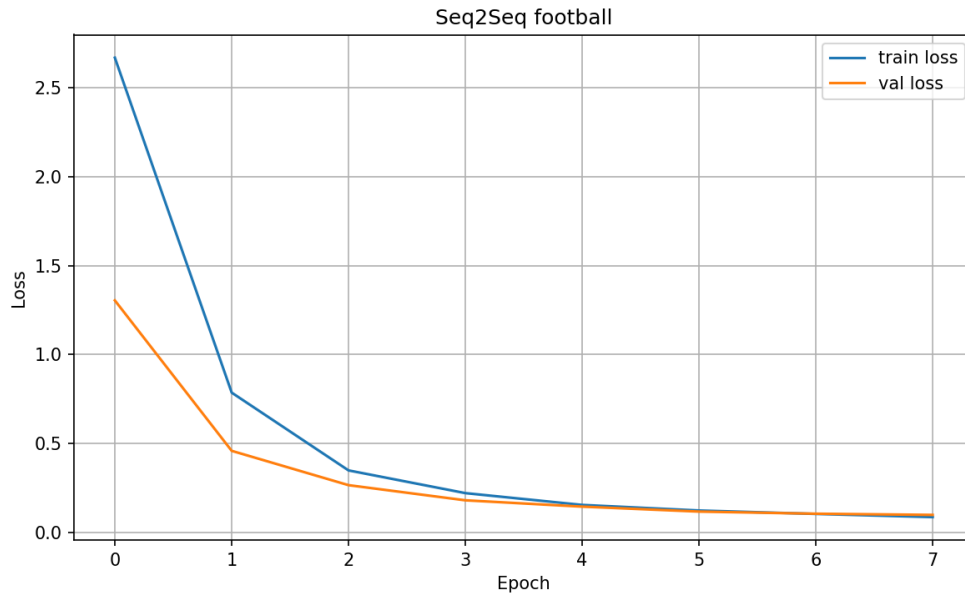


FIGURE 2 – Courbes d’entraînement du modèle Seq2Seq GRU

III.5 – Évaluation BLEU et Stratégies de Décodage

La métrique BLEU compare les n-grammes de la phrase générée avec ceux de la référence. Le greedy decoding choisit le token le plus probable à chaque étape, tandis que le beam search conserve plusieurs hypothèses partielles. Dans cette expérience, les scores BLEU sont élevés car les phrases sont courtes, régulières et construites à partir de patrons répétitifs.

TABLEAU 3 – Scores BLEU moyens du modèle Seq2Seq

Stratégie	Score BLEU moyen
Greedy decoding	97.35%
Beam search	97.03%

Analyse et interprétation

Le score BLEU très élevé s’explique par la nature contrôlée du corpus : les phrases sont courtes, régulières et construites à partir de patrons répétitifs. Le greedy decoding est légèrement meilleur que le beam search dans cette expérience, car les séquences cibles sont simples et ne nécessitent pas une exploration profonde de plusieurs hypothèses. Sur des phrases naturelles plus longues, le beam search pourrait devenir plus avantageux.

III.6 – Synthèse du Chapitre III

- Les modèles récurrents exploitent l’ordre des tokens grâce à un état caché.
- Le BPTT permet l’entraînement mais peut produire des gradients instables.
- LSTM et GRU améliorent la mémorisation grâce aux portes.

- Le Seq2Seq illustre la génération conditionnelle de texte.
- Les résultats élevés sont liés à un corpus footballistique structuré et doivent être interprétés avec prudence.

Chapitre IV – Comparaison Croisée et Conclusion Générale

IV.1 – Tableau Comparatif Global des Trois Architectures

TABEAU 1 – Comparaison globale des architectures étudiées dans FootAI

Critère	MLP	CNN	RNN / GRU / Seq2Seq
Type de données	Tabulaires	Images / grilles 2D	Séquences textuelles
Biais inductif	Aucun biais spatial ou temporel explicite	Localité, partage des poids, hiérarchie visuelle	Mémoire temporelle, ordre des tokens
Dataset utilisé	StatsBomb tirs	Roboflow YOLO crops	StatsBomb textuel
Meilleure performance	F1 rechargé : 56.71%	F1 : 86.10%	BLEU greedy : 97.35%
Limite principale	Dépendance aux variables construites	Confusion visuelle entre classes proches	Corpus trop régulier, généralisation à vérifier
Évolution naturelle	Feature engineering, TabNet, FT-Transformer	ResNet, EfficientNet, Vision Transformer	Attention, Transformers, modèles pré-entraînés

IV.2 – Le Biais Inductif : Clé de la Réussite en Deep Learning

La comparaison montre que la performance d'un modèle dépend de son adéquation avec la structure des données. Le MLP est raisonnable pour les tableaux mais ignore les relations spatiales. Le CNN exploite la géométrie des images et améliore nettement les résultats par rapport à un MLP appliqué à des pixels aplatis. Le RNN, le LSTM et le GRU introduisent une mémoire séquentielle indispensable pour le texte.

Cette idée rejoint le principe selon lequel aucun algorithme n'est optimal pour tous les problèmes. Le bon modèle est celui dont les hypothèses correspondent le mieux aux régularités de la donnée. Dans FootAI, les expériences illustrent cette idée de manière concrète : le CNN surpasse le MLP sur images, tandis que les modèles récurrents conviennent mieux aux séquences.

IV.3 – Perspectives d'Amélioration

Plusieurs pistes peuvent améliorer le projet :

- enrichir la partie tabulaire avec des variables métier plus fortes : angle, distance, pression défensive, minute du match ;
- augmenter le dataset image et appliquer de la data augmentation pour améliorer la robustesse du CNN ;
- utiliser un modèle CNN pré-entraîné ou un Vision Transformer pour comparer avec les CNN simples ;
- remplacer le Seq2Seq récurrent par un modèle avec attention ;
- tester le système sur un corpus textuel plus naturel et moins contrôlé.

IV.4 – Conclusion Générale

Ce projet a permis de mettre en pratique les principales familles d'architectures de Deep Learning étudiées dans le module. La partie MLP a montré l'importance de la préparation des données, de l'initialisation, de la gestion des paramètres et de la sauvegarde avec `state_dict`. La partie CNN a confirmé la pertinence des convolutions pour traiter des images, avec une amélioration nette par rapport au MLP image aplatie. La partie RNN/Seq2Seq a illustré la modélisation de séquences, l'intérêt des cellules récurrentes et les stratégies de décodage.

La conclusion principale est que le Deep Learning doit adapter son architecture à la nature de la donnée. Les données tabulaires, les images et les séquences ne possèdent pas la même géométrie ni les mêmes dépendances. Utiliser le même paradigme supervisé ne signifie donc pas utiliser le même réseau. Le choix du bon biais inductif reste un facteur central de réussite.

Références Bibliographiques

1. Rumelhart, D. E., Hinton, G. E., Williams, R. J. (1986). *Learning representations by back-propagating errors*. Nature.
2. Cybenko, G. (1989). *Approximation by superpositions of a sigmoidal function*. Mathematics of Control, Signals and Systems.
3. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE.
4. Hochreiter, S., Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation.
5. Cho, K. et al. (2014). *Learning phrase representations using RNN encoder-decoder for statistical machine translation*. EMNLP.
6. Glorot, X., Bengio, Y. (2010). *Understanding the difficulty of training deep feedforward neural networks*. AISTATS.
7. Kingma, D. P., Ba, J. (2015). *Adam : A method for stochastic optimization*. ICLR.
8. Ioffe, S., Szegedy, C. (2015). *Batch Normalization : Accelerating deep network training by reducing internal covariate shift*. ICML.
9. Papineni, K. et al. (2002). *BLEU : a method for automatic evaluation of machine translation*. ACL.
10. Paszke, A. et al. (2019). *PyTorch : An imperative style, high-performance deep learning library*. NeurIPS.